15 Interfaces web

Creación de una Aplicación Web con Flask para Calcular la Rentabilidad de Pizzas

El objetivo es crear una aplicación web que permita al usuario ingresar los diámetros y precios de dos pizzas, el ancho del borde, calcular la rentabilidad (costo por cm² con y sin bordes) y mostrar los resultados, con validación de datos y manejo de errores. Explicaremos tanto la programación del backend como el diseño estético del frontend, asegurando que sea responsivo.

Creación de una Aplicación Web con Flask para Calcular la Rentabilidad de Pizzas

Requisitos previos

- **Python**: Versión 3.6 o superior.
- **Flask**: Instalar con pip install flask.
- **Conocimientos básicos**: Python, HTML, CSS, JavaScript y conceptos de Flask.
- **Bootstrap 5**: Usaremos el CDN para un diseño moderno sin necesidad de instalarlo localmente.
- Un editor de texto (PyCharm) y un navegador web.

Objetivo

Crear una aplicación web que:

1. Permite al usuario ingresar el diámetro y precio de dos pizzas (familiar y mediana) y el ancho del borde.
2. Calcule el costo por cm² con y sin bordes, mostrando cuál pizza es más rentable.
3. Valide las entradas y maneje errores (valores no numéricos, negativos, etc.).
4. Tenga un diseño moderno, atractivo y responsivo usando Bootstrap 5.
5. Es fácil de usar en dispositivos móviles y de escritorio.

15 Interfaces web

Paso 1: Estructura del proyecto

Organizaremos el proyecto en una estructura clara:

```
pizza_calculator_web/
├── app.py
├── static/
│   ├── css/
│   │   └── styles.css
│   └── js/
│       └── scripts.js
├── templates/
│   └── index.html
└── requirements.txt
```

- ➢ app.py: Código del backend con Flask.
- ➢ static/css/styles.css: Estilos personalizados para complementar Bootstrap.
- ➢ static/js/scripts.js: JavaScript para validación en el frontend.
- ➢ templates/index.html: Plantilla HTML para la interfaz.
- ➢ requirements.txt: Dependencias del proyecto.

Paso 2: Configuración del entorno

1. **Crea el directorio del proyecto**:
   a. mkdir pizza_calculator_web
   b. cd pizza_calculator_web
2. **Crea un entorno virtual** (opcional, pero recomendado)
3. **Instala Flask**: pip install flask (o con PyCharm escribes flas)
4. **Crea requirements.txt**: text flask Puedes regenerarlo con pip freeze > requirements.txt después de instalar Flask.

Paso 3: Código del backend (app.py)

15 Interfaces web

El backend manejará las solicitudes, realizará los cálculos y devolverá los resultados. A continuación, el código para [app.py](app.py):

```
from flask import Flask, render_template, request, jsonify
import math

app = Flask(__name__)

def validate_inputs(diam_fam, price_fam, diam_med, price_med, border):
    try:
        diam_fam = float(diam_fam)
        price_fam = float(price_fam)
        diam_med = float(diam_med)
        price_med = float(price_med)
        border = float(border)

        if diam_fam <= 0 or price_fam <= 0 or diam_med <= 0 or price_med <= 0:
            return None, "Los diámetros y precios deben ser mayores que cero."
        if border < 0:
            return None, "El ancho del borde no puede ser negativo."
        if border * 2 >= diam_fam or border * 2 >= diam_med:
            return None, "El borde es demasiado ancho para el diámetro de la pizza."

        return (diam_fam, price_fam, diam_med, price_med, border), None
    except ValueError:
        return None, "Por favor, ingrese valores numéricos válidos."

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/calculate', methods=['POST'])
def calculate():
    # Obtener datos del formulario
    diam_fam = request.form.get('diam_familiar')
    price_fam = request.form.get('price_familiar')
    diam_med = request.form.get('diam_mediana')
    price_med = request.form.get('price_mediana')
    border = request.form.get('border_width', '2.0')  # Valor por defecto: 2 cm

    # Validar entradas
    inputs, error = validate_inputs(diam_fam, price_fam, diam_med, price_med, border)
    if error:
        return jsonify({'error': error})

    diam_fam, price_fam, diam_med, price_med, border = inputs

    # Calcular áreas totales
    area_fam = math.pi * (diam_fam / 2) ** 2
    area_med = math.pi * (diam_med / 2) ** 2

    # Calcular costos por cm² (con bordes)
    cost_per_cm2_fam = price_fam / area_fam
    cost_per_cm2_med = price_med / area_med
```

```
    # Calcular áreas sin bordes
    diam_fam_no_border = diam_fam - 2 * border
    diam_med_no_border = diam_med - 2 * border
    area_fam_no_border = math.pi * (diam_fam_no_border / 2) ** 2 if diam_fam_no_border >
0 else 0
    area_med_no_border = math.pi * (diam_med_no_border / 2) ** 2 if diam_med_no_border >
0 else 0

    # Calcular costos por cm² (sin bordes)
    cost_per_cm2_fam_no_border = price_fam / area_fam_no_border if area_fam_no_border >
0 else float('inf')
    cost_per_cm2_med_no_border = price_med / area_med_no_border if area_med_no_border >
0 else float('inf')

    # Determinar cuál es más rentable
    most_profitable = "Familiar" if cost_per_cm2_fam < cost_per_cm2_med else "Mediana"
    most_profitable_no_border = "Familiar" if cost_per_cm2_fam_no_border <
cost_per_cm2_med_no_border else "Mediana"

    # Formatear resultados
    result = {
        'area_fam': round(area_fam, 2),
        'cost_per_cm2_fam': round(cost_per_cm2_fam, 5),
        'area_med': round(area_med, 2),
        'cost_per_cm2_med': round(cost_per_cm2_med, 5),
        'most_profitable': most_profitable,
        'area_fam_no_border': round(area_fam_no_border, 2),
        'cost_per_cm2_fam_no_border': round(cost_per_cm2_fam_no_border, 5) if
area_fam_no_border > 0 else 'N/A',
        'area_med_no_border': round(area_med_no_border, 2),
        'cost_per_cm2_med_no_border': round(cost_per_cm2_med_no_border, 5) if
area_med_no_border > 0 else 'N/A',
        'most_profitable_no_border': most_profitable_no_border,
        'border': border
    }

    return jsonify({'result': result})

if __name__ == '__main__':
    app.run(debug=True)
```

Explicación del backend

- **Flask Setup**: Creamos una aplicación Flask y definimos dos rutas:
  - /: Renderiza la plantilla index.html.
  - /calculate: Procesa el formulario enviado vía POST y devuelve los resultados como JSON.
- **Validación**: La función validate_inputs comprueba que las entradas sean numéricas, positivas y que el borde no sea demasiado ancho.

- **Cálculos**: Calcula las áreas, costos por cm² y determina la pizza más rentable, devolviendo los resultados en un diccionario JSON.
- **Manejo de errores**: Si hay un error de validación, se devuelve un mensaje de error en el JSON.

Paso 4: Diseño del frontend (templates/index.html)

El frontend usa **Bootstrap 5** para un diseño moderno y responsivo, con validación en JavaScript para mejorar la experiencia del usuario.

Bootstrap 5 es un framework front-end de código abierto para desarrollar interfaces web responsivas y móviles de forma rápida y eficiente. Basado en HTML, CSS y JavaScript, ofrece componentes predefinidos y personalizables como botones, formularios, barras de navegación, modales y más, junto con un sistema de rejilla (grid) flexible para diseños adaptativos.

Características principales:

- **Sin jQuery**: Usa JavaScript puro, reduciendo dependencias.
- **Sistema de rejilla**: Basado en flexbox, con 12 columnas y breakpoints para diferentes tamaños de pantalla.
- **Componentes mejorados**: Incluye nuevos elementos como acordeones y mejoras en modales y tooltips.
- **CSS personalizado**: Variables CSS y soporte para temas personalizados.
- **Utilidades**: Clases para márgenes, paddings, colores, etc., que agilizan el desarrollo.
- **Compatibilidad**: Soporta navegadores modernos y diseños responsivos.

Para usarlo, puedes incluirlo vía CDN (usar un repositorio de descarga en internet) o instalarlo con npm (instalación en local). Ejemplo con CDN:

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
```

15 Interfaces web

Es ideal para prototipos rápidos o proyectos completos, con documentación clara en getbootstrap.com.

Creamos templates/index.html:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calculadora de Rentabilidad de Pizzas</title>
    <!-- Bootstrap CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
    <!-- Estilos personalizados -->
    <link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css')
}}">
</head>
<body>
    <div class="container mt-4">
        <h1 class="text-center mb-4">Calculadora de Rentabilidad de Pizzas</h1>
        <div class="card shadow-sm">
            <div class="card-body">
                <form id="pizzaForm">
                    <h3 class="card-title">Pizza Familiar</h3>
                    <div class="row mb-3">
                        <div class="col-md-6">
                            <label for="diam_familiar"
class="form-label">Diámetro (cm):</label>
                            <input type="number" class="form-control"
id="diam_familiar" name="diam_familiar" step="0.1" required>
                        </div>
                        <div class="col-md-6">
                            <label for="price_familiar"
class="form-label">Precio (€):</label>
                            <input type="number" class="form-control"
id="price_familiar" name="price_familiar" step="0.01" required>
                        </div>
                    </div>
                    <h3 class="card-title mt-4">Pizza Mediana</h3>
                    <div class="row mb-3">
                        <div class="col-md-6">
                            <label for="diam_mediana"
class="form-label">Diámetro (cm):</label>
                            <input type="number" class="form-control"
id="diam_mediana" name="diam_mediana" step="0.1" required>
                        </div>
                        <div class="col-md-6">
```

```
                            <label for="price_mediana" class="form-label">Precio
(€):</label>
                            <input type="number" class="form-control"
id="price_mediana" name="price_mediana" step="0.01" required>
                        </div>
                    </div>
                    <div class="row mb-3">
                        <div class="col-md-6">
                            <label for="border_width" class="form-label">Ancho
del borde (cm):</label>
                            <input type="number" class="form-control"
id="border_width" name="border_width" value="2.0" step="0.1" required>
                        </div>
                    </div>
                    <button type="submit" class="btn btn-primary w-100">Calcular
Rentabilidad</button>
                </form>
                <div id="error" class="alert alert-danger mt-3 d-none"></div>
                <div id="result" class="mt-3 d-none">
                    <h4>Resultados</h4>
                    <div id="result-content"></div>
                </div>
            </div>
        </div>
    </div>

    <!-- Bootstrap JS y Popper -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.j
s"></script>
    <!-- JavaScript personalizado -->
    <script src="{{ url_for('static', filename='js/scripts.js') }}"></script>
</body>
</html>
```

Explicación del HTML

- **Bootstrap 5**: Incluimos el CDN para CSS y JS, para poder dar un diseño moderno y componentes responsivos.
- **Estructura**:
  - Un contenedor (container) centra el contenido.
  - Un card con sombra para un look moderno.
  - Un formulario con campos para diámetros, precios y ancho del borde, organizados en filas responsivas (row, col-md-6).
  - Un botón de envío y áreas para mostrar errores (alert) y resultados.

- **Responsividad**: Uso de clases de Bootstrap como col-md-6 para que los campos se apilen en pantallas pequeñas.
- **Atributos HTML5**: type="number", step para decimales, y required para validación básica.

Paso 5: Estilos personalizados (static/css/styles.css)

Creamos estilos adicionales para mejorar la estética:

```
body {
    background-color: #f8f9fa;
    font-family: 'Arial', sans-serif;
}

h1 {
    color: #343a40;
    font-weight: bold;
}

.card {
    border-radius: 10px;
    background-color: #ffffff;
}

.form-label {
    font-weight: 500;
    color: #495057;
}

.btn-primary {
    background-color: #007bff;
    border-color: #007bff;
    transition: background-color 0.3s ease;
}

.btn-primary:hover {
    background-color: #0056b3;
    border-color: #0056b3;
}

.alert {
    border-radius: 8px;
}
```

```
#result {
    background-color: #e9ecef;
    padding: 15px;
    border-radius: 8px;
}

@media (max-width: 576px) {
    .card {
        margin: 10px;
    }
    h1 {
        font-size: 1.5rem;
    }
}
```

Explicación de los estilos:

- **Fondo y tipografía**: Fondo claro (#f8f9fa) y fuente sans-serif para un look limpio.
- **Card**: Bordes redondeados y fondo blanco para destacar el formulario.
- **Botón**: Color azul vibrante con transición suave al pasar el ratón.
- **Resultados**: Fondo gris claro para el área de resultados.
- **Media query**: Ajusta márgenes y tamaño de fuente para pantallas pequeñas.

Paso 6: JavaScript para validación y manejo de resultados (static/js/scripts.js)

El JavaScript valida el formulario en el frontend (en el navegador), comprobando que el valor introducido es válido, mostrando un mensaje de error si no lo es. También maneja la respuesta del servidor, la función showResult, mostrando el resultado de forma más visual:

```
document.getElementById('pizzaForm').addEventListener('submit', function(event)
{
    event.preventDefault();

    // Obtener valores del formulario
    const formData = new FormData(this);

    // Validación básica en el frontend
    const inputs = ['diam_familiar', 'price_familiar', 'diam_mediana',
'price_mediana', 'border_width'];
    let valid = true;
    inputs.forEach(input => {
        const value = formData.get(input);
        if (!value || isNaN(value) || value < 0) {
            valid = false;
        }
    });

    if (!valid) {
        showError('Por favor, complete todos los campos con valores numéricos no
negativos.');
        return;
    }

    // Enviar solicitud al servidor
    fetch('/calculate', {
        method: 'POST',
        body: formData
    })
    .then(response => response.json())
    .then(data => {
        if (data.error) {
            showError(data.error);
        } else {
            showResult(data.result);
        }
    })
    .catch(error => {
        showError('Error en el servidor. Inténtelo de nuevo.');
        console.error(error);
    });
});

function showError(message) {
    const errorDiv = document.getElementById('error');
    errorDiv.textContent = message;
    errorDiv.classList.remove('d-none');
    document.getElementById('result').classList.add('d-none');
```

```
}

function showResult(result) {
    const resultDiv = document.getElementById('result-content');
    resultDiv.innerHTML = `
        <h5>Con bordes incluidos:</h5>
        <p>Área Familiar: ${result.area_fam} cm², Costo por cm²:
${result.cost_per_cm2_fam} €/cm²</p>
        <p>Área Mediana: ${result.area_med} cm², Costo por cm²:
${result.cost_per_cm2_med} €/cm²</p>
        <p><strong>Pizza más rentable: ${result.most_profitable}</strong></p>
        <h5 class="mt-3">Sin bordes (borde de ${result.border} cm):</h5>
        <p>Área Familiar: ${result.area_fam_no_border} cm², Costo por cm²:
${result.cost_per_cm2_fam_no_border}</p>
        <p>Área Mediana: ${result.area_med_no_border} cm², Costo por cm²:
${result.cost_per_cm2_med_no_border}</p>
        <p><strong>Pizza más rentable:
${result.most_profitable_no_border}</strong></p>
        <h5 class="mt-3">Consideraciones:</h5>
        <ul>
            <li>Asegúrate de que la cantidad sea adecuada para evitar
desperdicio.</li>
            <li>Verifica si hay diferencias en ingredientes o promociones.</li>
        </ul>
    `;
    document.getElementById('result').classList.remove('d-none');
    document.getElementById('error').classList.add('d-none');
}
```

Explicación del JavaScript

- **Evento de formulario**: Captura el envío del formulario y previene el comportamiento predeterminado.
- **Validación en frontend**: Comprueba que los campos no estén vacíos y sean numéricos/no negativos.
- **Solicitud al servidor**: Usa fetch para enviar los datos del formulario a /calculate y manejar la respuesta JSON.
- **Manejo de errores**: Muestra mensajes de error en un div con clase alert-danger.
- **Mostrar resultados**: Formatea los resultados en HTML y los muestra en el div de resultados.

---

15 Interfaces web

Paso 7: Cómo ejecutar la aplicación

1. **Asegúrate de que la estructura esté completa**:
   a. Crea las carpetas static/css y static/js.
   b. Guarda app.py, index.html, styles.css y scripts.js en sus respectivas ubicaciones.
2. **Ejecuta el servidor**: python app.py
3. Accede a la aplicación:
   a. Abre un navegador y visita http://127.0.0.1:5000.
   b. Ingresa datos (e.g., 38 cm, 18 € para familiar; 29 cm, 13 € para mediana; 2 cm para borde).
   c. Haz clic en "Calcular Rentabilidad" para ver los resultados.

Paso 8: Diseño estético y responsividad

Estética

- **Bootstrap 5**: Proporciona un diseño limpio con componentes como card, form-control y btn-primary.
- **Colores**: Fondo claro (#f8f9fa), tarjeta blanca con sombra y botón azul vibrante para un look moderno.
- **Tipografía**: Uso de Arial (definido en CSS) para consistencia.
- **Transiciones**: Efecto suave en el botón al pasar el ratón.

Responsividad

- **Grid de Bootstrap**: Las clases col-md-6 aseguran que los campos se muestren en dos columnas en pantallas medianas o grandes y se apilen en móviles.
- **Media query en CSS**: Ajusta márgenes y tamaño de fuente para pantallas pequeñas (max-width: 576px).
- **Viewport meta tag**: <meta name="viewport" content="width=device-width, initial-scale=1.0"> asegura que la página se escale correctamente en dispositivos móviles.

Paso 9: Ejemplo de uso

Ingresa en el formulario:

- ➢ Pizza familiar: Diámetro = 38 cm, Precio = 18 €
- ➢ Pizza mediana: Diámetro = 29 cm, Precio = 13 €
- ➢ Ancho del borde: 2 cm

Resultado esperado (en el área de resultados):

```
Con bordes incluidos:
Área Familiar: 1134.11 cm², Costo por cm²: 0.01588 €/cm²
Área Mediana: 660.52 cm², Costo por cm²: 0.01967 €/cm²
Pizza más rentable: Familiar

Sin bordes (borde de 2.0 cm):
Área Familiar: 907.92 cm², Costo por cm²: 0.01983 €/cm²
Área Mediana: 490.87 cm², Costo por cm²: 0.02648 €/cm²
Pizza más rentable: Familiar
```

Consideraciones:

- ➢ Asegúrate de que la cantidad sea adecuada para evitar desperdicio.
- ➢ Verifica si hay diferencias en ingredientes o promociones.

Si ingresas un valor no numérico o negativo, aparecerá un mensaje de error en rojo.

Paso 10: Pruebas y depuración

- ➢ **Pruebas de funcionalidad**:
  - ○ Ingresa valores válidos y verifica que los cálculos coincidan con los de la versión Tkinter.
  - ○ Prueba con bordes de 0 cm para confirmar que los resultados con y sin bordes son idénticos.
- ➢ **Pruebas de errores**:
  - ○ Deja campos vacíos → Debería mostrar un error en el frontend.
  - ○ Ingresa letras o valores negativos → Debería mostrar un mensaje de error específico.
  - ○ Ingresa un borde demasiado ancho → Debería mostrar "El borde es demasiado ancho para el diámetro de la pizza."
- ➢ **Pruebas de responsividad**:

- Abre la aplicación en un navegador móvil o reduce la ventana del navegador para verificar que los campos se apilen correctamente.

Paso 11: Posibles mejoras

1. **Validación en tiempo real**: Agregar event listeners en JavaScript para validar campos mientras el usuario escribe.
2. **Animaciones**: Usar CSS o librerías como Animate.css para transiciones suaves al mostrar resultados.
3. **Más pizzas**: Permitir comparar más de dos pizzas dinámicamente.
4. Despliegue: Publicar la aplicación en un servicio como Render o Heroku.
5. **Multilingüe**: Agregar soporte para otros idiomas con un selector de idioma.
6. **Gráficos**: Incluir un gráfico (e.g., con Chart.js) para visualizar las áreas o costos.

Paso 12: Despliegue (opcional)

Para hacer la aplicación accesible online:

- **Usa un servicio como Render**:
  - Crea un repositorio en GitHub con el proyecto.
  - Conecta Render a tu repositorio y configura un servicio web.
  - Especifica `requirements.txt` y el comando `gunicorn app:app` (instala `gunicorn` con `pip install gunicorn`).
- **Configura variables de entorno**:
  - Asegúrate de que el puerto sea dinámico (`app.run(port=int(os.environ.get('PORT', 5000)))`).
- **Prueba el enlace público** proporcionado por Render.

[Descarga de la App Web Pizza Calculator]